

SoftRole-HetNet: The Architecture, Explained

Sep 28, 2026 · @Akshat Guduru

Summary

SoftRole-HetNet is a capability-aware extension of binary HetNet. Every agent carries a continuous, stochastic **role** that is recomputed each step from its memory and capabilities. The role selects a convex mixture of K learned weight sets, which every communication layer then uses. Only n -bit messages pass between agents.

The research question it tests: does a rate-limited adaptive role, conditioning both the binary channel and the agent's own processing, improve frozen-policy recovery after team or capability changes, beyond capacity-matched direct conditioning on history and capabilities?

Scope. Capability changes are *known* to the agent: its capability vector κ is an input, so this studies adaptation, not fault diagnosis. Capture ability is fixed per agent; sensing may change. HetNet is a baseline. No claim is made that SoftRole-HetNet contains HetNet as a special case.

What changed after review

Review finding	Change	Grounding
An n -way Gumbel-Softmax is one-hot, not n bits	Each bit is its own Bernoulli variable with a straight-through gradient (section 5)	The difference of two Gumbels is Logistic, so a 2-way Gumbel-Max sample is $\text{Bernoulli}(\sigma(u))$
FiLM's effective matrices $D_\gamma W$ can only rescale rows of one shared W	A bank of K full weight sets, mixed by a gate on the simplex (section 4)	Convex combinations of full matrices carry no row-proportionality constraint
Additive GAT scores rank neighbours identically for a fixed receiver role	GATv2 scoring (section 6)	Brody et al. (2022) prove GAT attention is static and GATv2 is dynamic

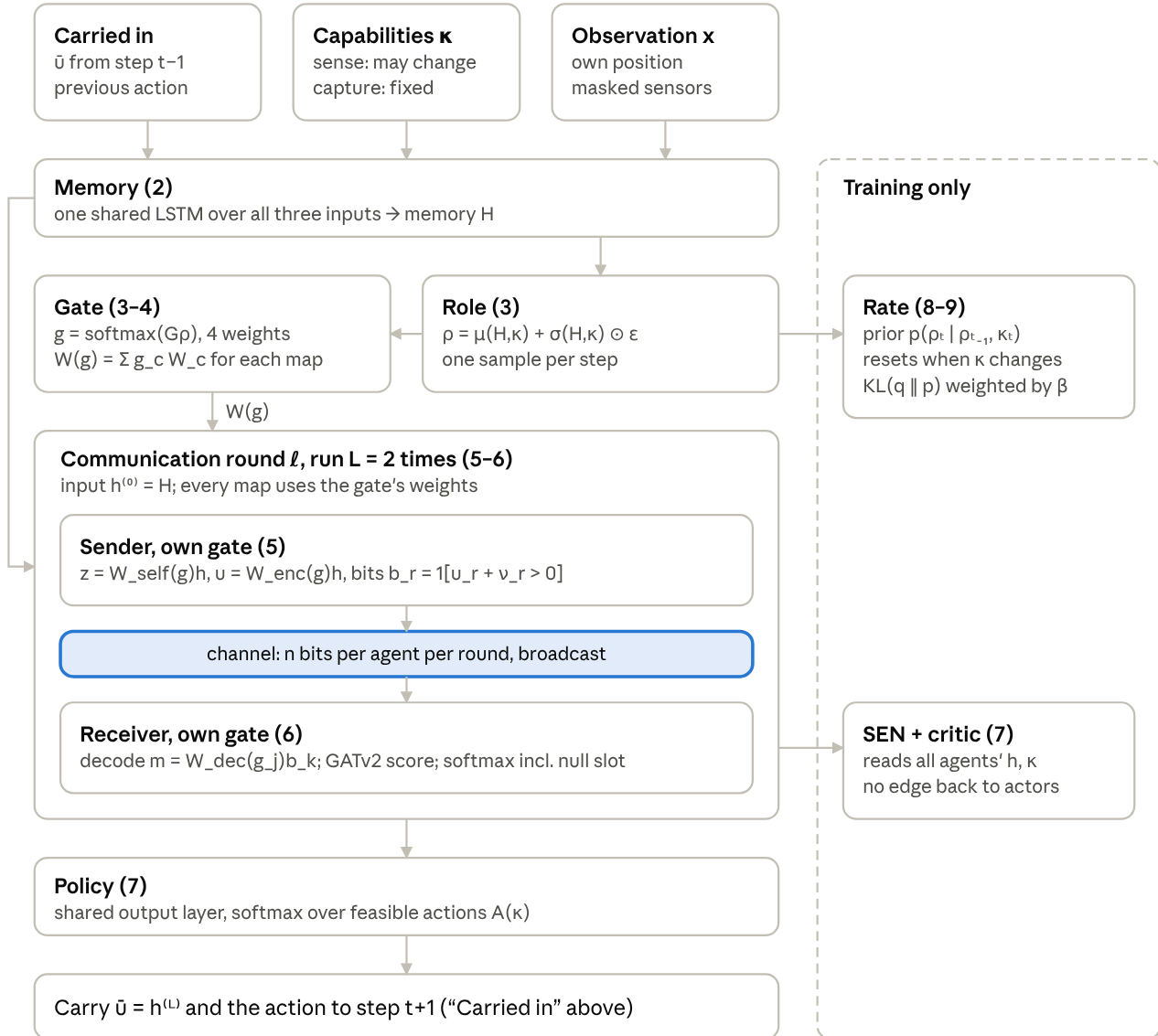
Review finding	Change	Grounding
A single softmax cannot ignore all messages or handle empty neighbourhoods	A null slot with a zero message (section 6)	The weights on real messages sum to $1 - \alpha_{\emptyset} \in (0, 1)$
The KL to a capability prior equals a rate $I(\rho; H \text{ given } \kappa)$ plus a marginal term, which is compression, not specialization	The prior becomes a persistence prior that resets on capability change; the KL is treated as an information rate with weight β ; free bits removed (sections 8–9)	The Hoffman–Johnson KL decomposition; KL-regularized RL with an information-asymmetric default (InfoBot)
The gradient estimator and baseline were unspecified	The reparameterized A2C estimator stated with its conditions and its two biases (section 10)	Common random numbers turn internal noise into exogenous state
KL and actor losses were aggregated differently	Every loss uses the same sum over agents and steps, divided by episodes (section 9)	Keeps regularization per agent-step fixed as team size changes
Time and layer indices were mixed	$H_{(j,t)}$ for memory, $h^{(\ell)}_{(j,t)}$ for communication layers, one role per step (sections 1–3)	Removes an ambiguity, not a mathematical change

All equations use HetNet’s notation where a HetNet counterpart exists; each section gives the HetNet equation it replaces.

At a glance

One role per step sets every layer's weights; only bits cross between agents

Agent j at step t . Numbers are section numbers. Dashed: training only.



SoftRole-HetNet forward pass for one agent at one step · $L = 2$ rounds, $K = 4$ weight sets

Read top to bottom. Memory produces one role per step; the role's gate selects the weights of every map in both communication rounds; only n -bit messages cross between agents; and the final features are carried into the next step. The dashed column exists only in training: the information-rate penalty and the critic.

1. Setting and notation

The team is a Dec-POMDP whose membership can change. At step t the active agents form a set $\mathcal{A}_t$, and the communication graph $\mathcal{G}_t = (\mathcal{A}_t, \mathcal{E}_t)$ is rebuilt from positions exactly as in HetNet: an edge $k \rightarrow j$ exists when j is within the sender's range.

Capabilities

Each agent has a capability vector, observed by the agent itself:

$$\kappa_{j,t} = \left[\kappa^{\text{sense}}_{j,t}, \kappa^{\text{cap}}_{j,t} \right] \in \{0, 1\}^2$$

The capture flag is fixed per agent; the sensing flag may change during an episode (the final experiment phase). HetNet's two classes are two values of κ : a predator is $(1, 0)$ and a PCP capture agent is $(0, 1)$, since capture agents are blind. A predator whose sensor fails becomes $(0, 0)$, a combination HetNet cannot represent.

Actions

All agents share one vocabulary $\mathcal{A} = \{\text{up, right, down, left, stay, capture}\}$; the PCP environment already uses these six actions for everyone. The feasible set is

$$\mathcal{A}(\kappa) = \begin{cases} \mathcal{A} & \text{if } \kappa^{\text{cap}} = 1 \\ \mathcal{A} \setminus \{\text{capture}\} & \text{if } \kappa^{\text{cap}} = 0 \end{cases}$$

The mask encodes physical feasibility. No role can grant an action the hardware lacks.

Observations

$$x_{j,t} = \left[s_{j,t}, o_{j,t}, \kappa^{\text{sense}}_{j,t} \right]$$

- $s_{j,t}$ is the agent's own grid position, which is proprioceptive and therefore local.
- $o_{j,t}$ is the local window of cell contents, zeroed when the sensor is unavailable.
- The trailing flag separates "the sensor reported nothing" from "there is no sensor". The PCP environment encodes the same information with a -1 fill in blind agents' class channels (its code also leaks that fill into teammates' windows; see the implementation guide, section 7).

What an agent may use at execution

$$\mathcal{I}^{\{\text{exec}\}}_{j,t} = \big\{ \{x_{j,\leq t}, \kappa_{j,\leq t}, a_{j,<t}, b_{k,\leq t}^{\{\ell\}} \mid k \in \mathcal{E}_{\leq t}\} \big\}$$

Its own inputs, its own past actions and the bits it has received. Nothing else, and in particular nothing computed by the state summary node, may enter an actor's forward pass. Because κ is given rather than inferred, the experiments study **adaptation to known capability changes**, not fault diagnosis.

Indices and symbols

Symbol	Meaning
j, k	An agent and one of its in-range neighbours (k sends, j receives)
t	Environment step
$\ell = 0, \dots, L-1$	Communication round within a step; $L = 2$, as in HetNet
$r = 1, \dots, n$	Bit index; $n = 16$ by default
$c = 1, \dots, K$	Index of a weight set in the bank; $K = 4$
$H_{-}(j,t), C_{-}(j,t)$	LSTM hidden and cell state: the agent's memory
$h^{\ell}_{-}(j,t)$	Features entering communication round ℓ ; $h^0 = H$
$\rho_{-}(j,t) \in \mathbb{R}^3$	The role; one sample per agent per step, reused by every round
$g_{-}(j,t) \in \Delta^{(K-1)}$	The gate: mixture weights over the K weight sets
u, b, m	Encoder logits, sent bits, decoded message
e_{jk}, α_{jk}	Attention score and weight; $\emptyset$ marks the null slot
$\eta(\kappa)$	Learned role embedding of a capability vector
λ_{ρ}, β	Role persistence and rate weight
ε, ν	Role noise (Gaussian) and bit noise (Logistic)

2. Memory

One LSTM, shared by every agent, summarizes each agent's history:

```

\begin{aligned}
&\text{HetNet:} \quad h_j = \text{LSTM}^{(i)}(\text{ReLU}(W^{(i)}_{\text{pre}} x_j)) \\
&\text{SoftRole:} \quad (H_{j,t}, C_{j,t}) = \\
&\quad \text{LSTM}_{\theta}(\text{Pre}_{\theta}(x_{j,t}), \kappa_{j,t}, \bar{u}_{j,t-1}, \mathbf{e}(a_{j,t-1})) \\
&\quad (H_{j,t}, C_{j,t-1})
\end{aligned}

```

Input	Meaning	Why it is there
$\text{Pre}_{\theta}(x_{j,t}) = \text{ReLU}(W_{\text{pre}} x + b)$	Encoded observation	One encoder for all agents, since observations share a format
$\kappa_{j,t}$	Current capabilities	Lets memory register when a capability changed
$\bar{u}_{j,t-1} = h^{(L)}_{j,t-1}$	The agent's post-communication features from the previous step	Makes what the agent heard part of its state
$\mathbf{e}(a_{j,t-1})$	One-hot previous action	Standard in recurrent MARL agents (PyMARL, ROMA): with a stochastic policy, the agent must know what it did

Why $\bar{u}$ matters, precisely. In HetNet's code the LSTM reads only the agent's own inputs, so $H_{j,t}$ is a function of $x_{j,\leq t}$ alone. Its memory then carries zero information about past messages beyond what its own observations imply: $I(H_{j,t}; b_{k,<t} \text{ given } x_{j,\leq t}) = 0$. A blind capture agent forgets a report as soon as predators stop repeating it. Feeding $\bar{u}$ back makes that information part of the recurrent state.

Timing. The role at step t is computed from $H_{j,t}$, which contains messages only up to step $t - 1$. A teammate's message therefore changes the role one step later. The current action can still react to that message within step t , through the communication rounds of section 6.

Membership. An agent that spawns at step t^0_j starts from zero state: $H = C = \bar{u} = 0$ and a "stay" previous action. An agent that leaves is removed from  t , its state discarded, and its edges disappear.

Truncated backpropagation. HetNet cuts gradients through the recurrence every 5 steps (detach_gap). $\bar{u}$ is part of the recurrent state, so it must be detached at the same boundaries as H and C. Otherwise the computation graph grows with the episode, and gradient terms longer than the window are silently kept for $\bar{u}$ alone.

3. The role

Encoder

The role is a sample from a diagonal Gaussian computed from memory and capabilities (the ROMA idea, with ROMA's raw-observation input replaced by memory):

$$q_{\theta}(\rho_{j,t} \mid H_{j,t}, \kappa_{j,t}) = \mathcal{N}(\mu_{\theta}(H_{j,t}, \kappa_{j,t}), \text{diag}(\sigma^2_{\theta}(H_{j,t}, \kappa_{j,t}))), \quad \rho_{j,t} = \mu + \sigma \odot \epsilon_{j,t}, \quad \epsilon_{j,t} \sim \mathcal{N}(\mathbf{0}, I_3)$$

- The variance is parameterized as $\sigma^2 = \sigma^2_{\min} + \text{softplus}(\cdot)$. The floor keeps the KL of section 9 finite and the role noise bounded below.
- The sample is reparameterized, so gradients reach μ and σ by ordinary calculus (the Foundations guide, 1.8).
- **One sample per agent per step**, reused by every communication round. Sampling a fresh role per round would define a different model: L independent latents per step, with more noise and no single role to interpret.
- At execution the role is sampled, because the policy is defined that way. Using $p = \mu$ instead is an ablation.

Gate

$$g_{j,t} = \text{softmax}(\mathbf{G} \rho_{j,t} + \mathbf{g}_0) \in \Delta^{K-1}, \quad \mathbf{G} \in \mathbb{R}^{K \times 3}, \quad K = 4$$

Why the role is 3-dimensional when $K = 4$. Softmax ignores any constant added to all K logits, so its output depends on the logits only through the $(K - 1)$ -dimensional subspace orthogonal to $\mathbf{1}$. A role of dimension $K - 1 = 3$ parameterizes that subspace without redundancy when G's columns span it. A smaller role would reach only a lower-

dimensional surface inside the simplex; a larger one would add directions the gate cannot see, which the KL of section 9 would penalize for no benefit. Three dimensions also match ROMA and can be plotted directly.

What a role means, and what it does not

- ρ is a latent coordinate. $g(\rho)$ is a set of **mixture weights over K learned processing modes**: $g_{(j,t,c)}$ is the share of weight set c in agent j 's effective maps at step t (section 4).
- g is not a probability of holding a discrete role. "60% scout" is not implied. That reading would need a categorical latent, which this model does not have.
- Whether roles carry task-relevant structure is an empirical question, tested by interventions rather than assumed from plots of ρ (section 11).

4. The weight bank

Every role-dependent linear map in a communication round is a gate-weighted mixture of K learned affine maps:

$$W^{\{\phi\}}_{\text{ell}}(g) = \sum_{c=1}^K g_c \cdot W^{\{\phi\}}_{\text{ell},c}, \quad \text{quad} \\ b^{\{\phi\}}_{\text{ell}}(g) = \sum_{c=1}^K g_c \cdot b^{\{\phi\}}_{\text{ell},c}, \quad \text{quad } \phi \in \{ \text{self}, \text{enc}, \text{dec}, \text{att}, \mathbf{a}, \varnothing \}$$

Map ϕ	Whose gate selects it	Role in the round
self	The agent's own, g_j	Transforms its own features (section 6)
enc	The sender's, g_k	Turns features into bit logits (section 5)
dec	The receiver's, g_j	Turns received bits into features (section 6)
att, $\mathbf{a}$, $\varnothing$	The receiver's, g_j	Score each message and the null slot (section 6)

The LSTM, the observation encoder, the role encoder and the output layer are shared and not banked.

Computation. For an affine map, mixing weights equals mixing outputs: $W(g)h + b(g) = \sum_c g_c (W_c h + b_c)$. An implementation runs K small matrix products and takes a weighted sum; it never builds a per-agent matrix.

Why a bank rather than FiLM

	FiLM: $D_{\gamma}(\rho) W$	Bank: $\sum_c g_c W_c$
Set of effective matrices	Row rescalings of one shared W	The convex hull of K full matrices
Degrees of freedom from the role	d' row scales, all tied to W 's row directions	$K - 1$ mixture weights, directions unconstrained
The review's counterexample ($W_1 = I$, $W_2 =$ a permutation)	Not representable	Representable with $K \geq 2$
Size of effective weights	Unbounded as γ grows	$\ W(g)\ \leq \max_c \ W_c\ $ for any norm, by convexity
Parameters	1 matrix plus a small generator	K matrices

Neither family contains the other in general. The bank was chosen for two properties, both proved in one line. It escapes the row-proportionality restriction the review identified. And the triangle inequality bounds the effective weights by the largest expert, so role noise cannot inflate them. FiLM remains a cheaper ablation.

In HetNet's code, each round has 4 attention heads; each head has its own K weight sets, and the equations show one head. Measured in the reference implementation, the banked rounds hold 67,744 parameters against 13,040 in HetNet-Binary's agent-side layers, about 5 $\times$: $K = 4$ weight sets instead of 2 classes, plus GATv2's attention matrices. The whole network is still smaller, 233,513 parameters against 491,832, because HetNet's 625-wide observation layer alone holds 391,250. Capacity-matched baselines must account for that.

Two research-grade details

The experts must differ at initialization. If $W_1 = \dots = W_K$, every expert produces the same output, so $\partial \mathcal{L} / \partial g_c$ is the same number for every c . The softmax Jacobian $J = \text{diag}(g) - g g^T$ sends constant vectors to zero ($J\mathbf{1} = g - g(g^T \mathbf{1}) = 0$). Then no gradient reaches the gate, G or the role encoder through any banked map, and the role never learns. Independent Xavier initialization of each expert breaks the symmetry.

Expert collapse is a known failure of mixtures. Gates in mixture-of-experts models tend to concentrate on a few experts (Shazeer et al., 2017). If every $g_{(j,t)}$ settles on one expert, the model reduces to a shared-weight network. Monitor the average gate entropy

and each expert's load $E[g_c]$. Add a balancing penalty only if collapse is actually observed.

5. The binary channel

In each round, the sender's gate selects its encoder, and each of the n logits becomes its own bit:

```
\begin{aligned}
u_{k,\ell} &= W^{\{\text{enc}\}}_{\ell}(g_k) \setminus, h^{\{\ell\}}_k + \\
&b^{\{\text{enc}\}}_{\ell}(g_k) \in \mathbb{R}^n \setminus \\
b_{k,\ell,r} &= \mathbb{1}\big[\setminus, u_{k,\ell,r} + \nu_{k,\ell,r} > 0 \setminus, \big], \\
&\quad \nu_{k,\ell,r} \overset{\{\text{iid}\}}{\sim} \operatorname{Logistic}(0, \\
&\quad 1), \quad r = 1, \ldots, n \\
\end{aligned}
```

Each bit is exactly Bernoulli($\sigma(u_r)$). The Logistic CDF is the sigmoid, so $P(b_r = 1) = P(v_r > -u_r) = 1 - \sigma(-u_r) = \sigma(u_r)$. This is also the two-way Gumbel-Max trick with logits $(0, u_r)$, because the difference of two independent standard Gumbels is standard Logistic. The message $b_{(k,\ell)} \in \{0,1\}^n$ can take 2^n values, and it is literally what crosses the channel, in training as well as execution.

HetNet's code computes the same thing: its binarizer maps each logit z to two logits with `nn.Linear(1, 2)`, which gives $P(\text{bit} = 1) = \sigma(kz + k_0)$. Here the encoder's own weights and bias already provide that slope and offset, so the extra layer is redundant.

Gradient: straight-through

```
\frac{\partial b_{k,\ell,r}}{\partial u_{k,\ell,r}} \setminus; \setminus; \frac{\partial}{\partial u} \setminus, \sigma\Big(\frac{u_{k,\ell,r} + \nu_{k,\ell,r}}{T}\Big), \\
\quad T = 1
```

The true derivative of a threshold is zero almost everywhere, so training substitutes the derivative of the relaxed sample, as HetNet does. This estimator is biased (Bengio et al., 2013), and the bias should be stated in any write-up. It is one of the two biases listed in section 10.

What crosses the channel, and how much

- One n -bit payload per sender per round, broadcast to every in-range neighbour.

- Nothing else: decoding and attention are computed by the receiver from the bits it received. Roles are never transmitted.
- Bits per step for the whole team, with broadcast counting:

$$B_t = L \cdot n \cdot |\mathcal{N}_t| \cdot \big(\text{point-to-point counting} + L \cdot n \cdot |\mathcal{E}_t|\big)$$

With $L = 2$ and $n = 16$, that is 32 bits per agent per step, the same budget as HetNet-Binary. Soft attention does not reduce B_t : a message with a low weight was still sent.

At evaluation the bits are sampled, since that is the channel the policy was trained with; HetNet's code does the same. Thresholding at $u > 0$ gives a deterministic channel and is worth reporting as a variant.

6. Receiving: decode, attend, update

In round ℓ , receiver j uses its own gate g_j for everything it computes:

```
\begin{aligned}
z_j &= W^{\{\text{self}\}}_{\ell}(g_j), h^{\{\ell\}}_j + b^{\{\text{self}\}}_{\ell}(g_j) \\
m^{\{jk\}}_{\ell} &= W^{\{\text{dec}\}}_{\ell}(g_j), b_{k,\ell} + \\
&b^{\{\text{dec}\}}_{\ell}(g_j) \\
e_{\{jk\}} &= \mathbf{a}_{\ell}(g_j)^{\text{top}} \operatorname{LeakyReLU}\big( \\
&W^{\{\text{att}\}}_{\ell}(g_j), [\, z_j \, , \, \text{Vert}\, , \, m^{\{jk\}}_{\ell} \, , \, ] \big), \quad \\
e_{\{j\varnothing\}} &= w^{\{\varnothing\}}_{\ell}(g_j) \\
\alpha_{\{jk\}} &= \frac{\exp(e_{\{jk\}})}{\exp(e_{\{j\varnothing\}}) + \sum_{k' \in N_t(j)} \exp(e_{\{jk'\}})}, \quad M_j = \sum_{k \in N_t(j)} \alpha_{\{jk\}}, \\
m^{\{jk\}}_{\ell} & \\
h^{\{\ell+1\}}_j &= \operatorname{ReLU}\big(z_j + M_j\big) \, ; \, ; \, (\ell < L-1), \\
&\quad h^{\{L\}}_j = z_j + M_j \\
\end{aligned}
```

Line	HetNet (Eqs. 2–4, binary code)	SoftRole-HetNet
Self transform	$\omega_i h_j$, one matrix per class	Bank selected by the agent's own gate
Decoding	$\omega_i^{\text{dec}} b_k$, one decoder per receiving class	Bank selected by the receiver's gate

Line	HetNet (Eqs. 2–4, binary code)	SoftRole-HetNet
Score	$\text{LeakyReLU}(\mathbf{a}_{\text{src}}^T \mathbf{m} + \mathbf{a}_{\text{dst}}^T \boldsymbol{\omega}_i \mathbf{h}_j)$, one vector per edge type	GATv2 form, parameters from the receiver's gate
Normalization	One softmax per sender class; total weight = number of neighbour classes present	One softmax over all neighbours plus the null slot; total weight on messages = $1 - \alpha_{j\emptyset}$
Update	ReLU in hidden rounds, none in the last	Same

Each round has 4 heads, concatenated in hidden rounds and averaged in the last, as in HetNet.

Why GATv2 scoring

For a fixed receiver, HetNet's score is $\text{LeakyReLU}(s_j + t_k)$, with t_k depending only on the message. LeakyReLU is increasing, so every receiver state ranks the neighbours by t_k : attention is **static**. GATv2 applies the nonlinearity before the final projection, and Brody et al. (2022) prove this makes attention dynamic: different queries can prefer different keys.

A two-unit example shows it. Take scalar z and m , $\mathbf{W}^{\text{att}} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$ and $\mathbf{a} = (-1, -1)$. Then $e = -[\text{LeakyReLU}(z - m) + \text{LeakyReLU}(m - z)] = -0.8 |z - m|$, so the receiver attends most to the message closest to its own state, whichever that is. No additive score can express this.

Why the null slot

A null candidate with a learned score and a zero message is added to every softmax. Each property below follows directly from the formula for α :

- **Listening to nothing is possible.** The weights on real messages sum to $1 - \alpha_{j\emptyset} \in (0, 1)$. With a plain single softmax they would always sum to exactly 1.
- **Empty neighbourhoods are well defined.** If $N_t(j)$ is empty, $\alpha_{j\emptyset} = 1$ and $M_j = 0$ exactly, with no division by zero.
- **"Communication off" is a clean intervention.** Forcing $\alpha_{j\emptyset} = 1$ gives $M_j = 0$. Zeroing the bits would not: the decoder would still output its bias $b^{\text{dec}}(g_j)$.

- **Neighbour order is irrelevant.** M_j is a weighted sum over a set, with weights that depend only on (z_j, m^{jk}) . The round is therefore permutation-equivariant over agents, which the implementation guide turns into a unit test.

7. Policy, critic and the state summary node

Policy

One output layer, shared by all agents, followed by a feasibility mask:

$$\pi_{\theta}(a \mid \mathcal{I}^{\text{exec}}_{j,t}) = \frac{\mathbb{1}[a \in \mathcal{A}(\kappa_{j,t})] \exp(w_a^{\text{top}} h^{(L)}_{j,t} + c_a)}{\sum_{a' \in \mathcal{A}(\kappa_{j,t})} \exp(w_{a'}^{\text{top}} h^{(L)}_{j,t} + c_{a'})}$$

HetNet uses one output layer per class, of size 5 or 6. Sharing W_{out} does not make the policy role-independent: $h^{(L)}$ already depends on ρ through every banked map. In code the mask is added to the logits as $\log \mathbb{1}[\cdot]$, which is $-\infty$ for infeasible actions.

State summary node (training only)

One SEN replaces HetNet's one-per-class:

```
\begin{aligned}
h^{(0)}_s &= \varphi_t = \text{Big}[\, | \mathcal{N}_t |, \,; \, \textstyle\sum_j \kappa^{\text{sense}}_{j,t}, \,; \, \sum_j \kappa^{\text{cap}}_{j,t}, \,; \, \text{grid side} \, \, \text{Big} \, \, \\
h^{(\ell+1)}_s &= \text{ReLU}(\text{Big}(W_s, h^{(\ell)}_s + \sum_{j \in \mathcal{N}_t} \beta_{sj}, W_{2s}, \text{Big}[\, h^{(\ell)}_j \,; \,; \, \text{sg}(\rho_{j,t}) \,; \,; \, \kappa_{j,t} \, \text{Big}]) \\
\end{aligned}
```

$\beta_s \cdot$ is a softmax over agents, and $\text{sg}(\cdot)$ stops gradients. φ_t describes the team without assuming a fixed composition; HetNet's code feeds the constant $[\text{num}_P, \text{num}_A, \text{world_dim}, 500]$ instead.

Critic

A per-agent critic, HetNet's third design (Sec. 5.2), extended with the role and capabilities:

$$V_{j,t} = w_V^{\text{top}} \cdot \text{ReLU}\left(\sum_s w_{s,j} \cdot h_s^{(L)} + b_V \right) + b'_V$$

There is no per-class critic to fall back on, and rewards are per agent in the PCP code.

Two rules

1. **No path from the SEN into any actor.** There is no edge $s \rightarrow j$. Deleting the SEN and the critic must leave every action distribution unchanged, and that becomes a unit test.
2. **A role-dependent baseline is allowed.** V depends on the sampled p , which would bias a score-function gradient on p . The gradient here is pathwise instead, and section 10 shows that any baseline computed before the action is sampled keeps it unbiased.

8. The role process over time

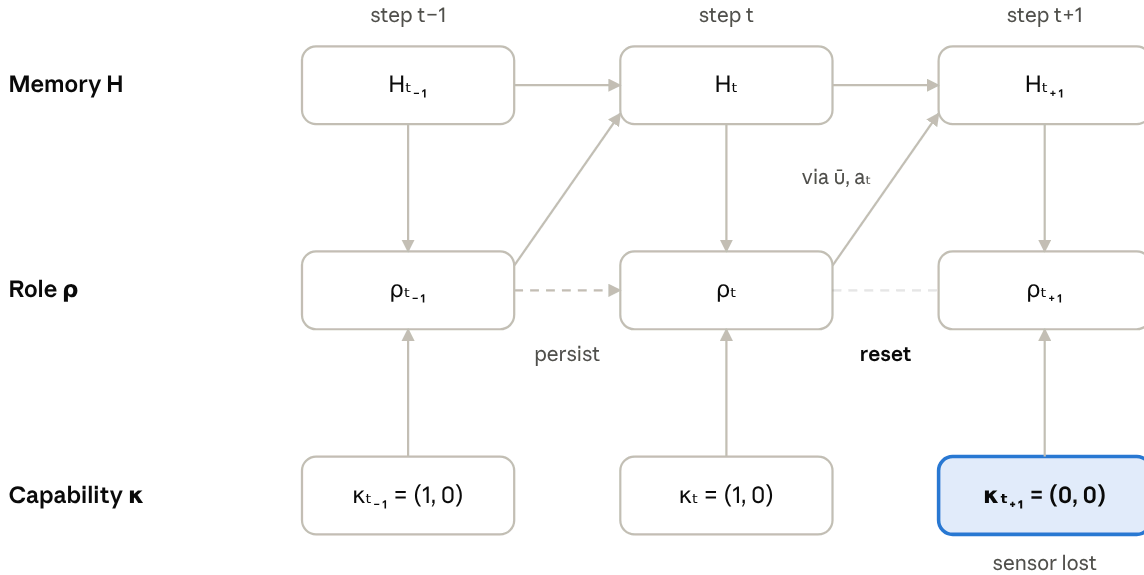
The prior over roles is a first-order process: persistent while capabilities are unchanged, and reset to a capability default at a spawn or a capability change.

```
\begin{aligned}
p(\rho_{j,t} \mid \rho_{j,t-1}, \kappa_{j,t}) &= \text{mathcal{N}}\Big(
\lambda_{j,t}, \rho_{j,t-1} + (1 - \lambda_{j,t})\eta(\kappa_{j,t}), \Sigma_{j,t} \\
(1 - \lambda_{j,t}^2)I \Big) \\
\lambda_{j,t} &= \begin{cases} \lambda_{\rho} & j \in \text{mathcal{N}}_{t-1} \text{ and } \\
& \kappa_{j,t} = \kappa_{j,t-1} \\
0 & \text{otherwise (spawn or} \\
& \text{capability change)} \end{cases} \\
\end{aligned}
```

$\eta(\kappa)$ is a learned embedding: the default role for a capability vector.

Roles persist from step to step; a capability change resets the prior

Solid arrows: computation. Dashed arrow: the prior's dependence on the previous role.



$$p(\rho_t \mid \rho_{t-1}, \kappa_t) = N(\lambda_t \rho_{t-1} + (1 - \lambda_t) \eta(\kappa_t), (1 - \lambda_t^2) I); \lambda_t = \lambda_{\rho}, \text{ or } 0 \text{ at a spawn or capability change}$$

The role process as a graphical model · three steps, a sensor lost at $t+1$

Each role is computed from memory and capability (solid arrows) and feeds the next step's memory through $\bar{u}$ and the action. The prior ties ρ_t to ρ_{t-1} (dashed), except at $t+1$: the sensor is lost, κ changes, and the prior resets to the default for the new capability.

Properties, each a one-line calculation

- **Stationary law.** With capabilities fixed, this is an AR(1) process with coefficient λ_{ρ} and innovation variance $1 - \lambda_{\rho}^2$. Its stationary distribution is $\mathbb{N}(\eta(\kappa), I)$, since the variance v solves $v = \lambda_{\rho}^2 v + 1 - \lambda_{\rho}^2$.
- **Resets are neutral.** At a reset the prior is $\mathbb{N}(\eta(\kappa_t), I)$, exactly the stationary law for the new capability. A capability change therefore costs no more than starting fresh with that capability.
- **Persistence timescale.** Prior autocorrelation at lag τ is λ_{ρ}^{τ} , with an e-folding time of $-1/\ln \lambda_{\rho}$ steps. For $\lambda_{\rho} = 0.9$ that is about 9.5 steps, against 80-step PCP episodes.
- **One knob.** Tying the innovation variance to $1 - \lambda_{\rho}^2$ fixes the scale of ρ at 1. The gate's matrix G can absorb any other scale, so nothing is lost, and λ_{ρ} is the only prior hyperparameter.
- **Nested special case.** With $\lambda_{\rho} = 0$ this is the static capability prior of the earlier proposal. The review's objection to that version is narrowed in section 9.

How this relates to ROMA. ROMA wants roles that are temporally stable, but its identifiability objective is identically zero under its own model (Part 3, 3.4 of the HetNet & ROMA guide). The persistence prior states stability directly as a probabilistic model, so it cannot vanish.

Gradients. $p_{\beta}(j, t-1)$ in the prior mean is the sampled, reparameterized previous role, so the rate term also trains the previous step's encoder output, as in sequential latent-variable models. That dependence is cut at the same truncated-backpropagation boundaries as the memory.

9. Training objective

What is maximized

KL-regularized reinforcement learning, with the role prior as the default distribution:

$$J_{\beta}(\theta) = \mathbb{E} \left[\sum_t \sum_{j \in \mathcal{N}_t} r_{j,t} \right] - \mathbb{E} \left[\sum_t \sum_{j \in \mathcal{N}_t} D_{\text{KL}}(q_{\theta}(\rho_{j,t} \mid H_{j,t}, \kappa_{j,t}) \parallel p(\rho_{j,t} \mid \rho_{j,t-1}, \kappa_{j,t})) \right]$$

What each update minimizes

For a batch of E on-policy episodes:

$$\begin{aligned} \mathcal{L} &= c_{\pi} \left(\mathcal{L}_{\pi} + \beta \right) + \mathcal{L}_V \\ \mathcal{L}_{\pi} &= -\frac{1}{E} \sum_e \sum_t \sum_{j \in \mathcal{N}_t} \log \pi_{\theta}(a_{j,t} \mid \cdot); \quad \text{sg}(\hat{A}_{j,t}) \\ \mathcal{L}_{\text{rate}} &= \frac{1}{E} \sum_e \sum_t \sum_{j \in \mathcal{N}_t} D_{\text{KL}}(q_{j,t} \parallel p_{j,t}) \\ \mathcal{L}_V &= \frac{1}{E} \sum_e \sum_j \text{mean}_t \left(V_{j,t} - R_{j,t} \right)^2 \end{aligned}$$

- **Same aggregation for policy and rate.** Both sum over agents and steps and divide by episodes, so β means nats of rate per unit of advantage-weighted log-likelihood per agent-step, whatever the team size. Summing one and averaging the other would silently strengthen regularization as teams grow.

- $c_\pi = 50$ and the critic's mean-over-time form follow HetNet's released code, with agents in place of classes.
- **Advantages.** GAE per agent with $\lambda_{\text{GAE}} = 0.95$ and HetNet's $\gamma = 1$; $R_-(j,t) = \hat{A}_-(j,t) + V_-(j,t)$ is the λ -return. $\hat{A}$ is normalized within each capture-capability group, because PCP rewards differ systematically by that flag (capture agents keep receiving the step penalty until they capture).

The rate term

With prior mean $\bar{m}_-(j,t)$ and prior variance $v_t = 1 - \lambda_-(j,t)^2$, the KL has a closed form, summed over the 3 role dimensions:

$$D_{\text{KL}}(q_{\{j,t\}} \parallel p_{\{j,t\}}) = \frac{1}{2} \sum_{r=1}^3 \left[\frac{\sigma_r^2 + (\mu_r - \bar{m}_r)^2}{v_t} - 1 + \ln \frac{v_t}{\sigma_r^2} \right]$$

Conditioning the reviewer's identity on $C = (\rho_{(t-1)}, \kappa_t)$ gives:

$$\mathbb{E}_{\rho_t, \kappa_t} [D_{\text{KL}}(q_{\{j,t\}} \parallel p_{\{j,t\}})] = \mathbb{E}_{\rho_t, \kappa_t} [D_{\text{KL}}(q_{\{j,t\}} \parallel p_{\{j,t\}} | \rho_{(t-1)}, \kappa_t)] + \mathbb{E}_{\rho_t, \kappa_t} [D_{\text{KL}}(\bar{q}_{\{j,t\}} \parallel p_{\{j,t\}} | \rho_{(t-1)}, \kappa_t)]$$

The proof adds and subtracts $\log \bar{q}$ inside the expectation. It needs ρ_t to be independent of $\rho_{(t-1)}$ given (H_t, κ_t) , which holds because ρ_t 's noise ε_t is fresh.

What β buys. The first term is the information a new role carries about history *beyond the previous role and current capability*: a switching cost. Holding a role steady is free, a capability-driven reset is free because the prior resets, and every other change must pay in return. This is KL-regularized RL with an information-asymmetric default (Galashov et al., 2019; Goyal et al., InfoBot, 2019). β is therefore swept, tracing a rate–return curve, not tuned to one value.

Which parameters each loss trains

Parameters	$\mathcal{L}_\pi$	$\mathcal{L}_{\text{rate}}$	$\mathcal{L}_V$
Role encoder (μ, σ) , gate (G, g_o)	Yes, through the reparameterized ρ	Yes	Excluded
Capability embedding η	No	Yes	No

Parameters	$\mathcal{L}_\pi$	$\mathcal{L}_\text{rate}$	$\mathcal{L}_V$
Observation encoder, LSTM	Yes	Yes, through $\mu(H)$	Yes
Banked experts, W_out	Yes	Only across steps, through $\bar{u}$	First-round experts, through the SEN
SEN, critic head	No	No	Yes

Excluding the critic from the role parameters is a design choice. With it, the role parameters follow the gradient of J_β exactly (up to the biases of section 10), so roles are shaped only by return and rate, not by whatever features predict value. The implementation takes the critic's gradient only with respect to the non-role parameters.

Defaults

Setting	Value	Source
K, role dimension	4, 3	Section 3
λ_ρ	0.9, an e-folding time of about 9.5 steps	Section 8
σ^2_min	0.01, a 0.1 standard-deviation floor on a unit-scale role	Keeps the KL finite
β	Swept on a log grid, including 0	This section
n, L, heads $\times$ width	16 bits, 2 rounds, 4×16	HetNet recipe
$c_\pi, \gamma, \lambda_\text{GAE}$	50, 1, 0.95	HetNet's released code
Optimizer	RMSprop, learning rate 10^{-4} , gradient clip 0.75, BPTT window 5	Your fork's reproduction recipe

10. The gradient estimator

Roles and bits are sampled before actions and influence several agents at once. So the gradient needs a precise statement: which estimator is used, why it is valid, and where it is biased.

Internal noise as exogenous state

Collect every internal random draw as $\xi = \{\epsilon_{(j,t)}, v_{(k,t,\ell,r)}\}$. These come from fixed distributions that do not depend on θ . Given ξ and the realized history, each agent's logits are a deterministic function $f_{(j,t)}(\theta; \xi, \tau_{(<t)})$, passing through its own role and gate and its neighbours' bits. The environment reacts only to actions.

Treat ξ as part of an augmented state that the environment ignores. The joint policy is then an ordinary stochastic policy over actions, factorizing across agents given the logits, and the policy-gradient theorem applies (the common-random-numbers idea behind PEGASUS, Ng and Jordan, 2000):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\xi, \tau} \left[\sum_t \sum_j \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{j,t} \mid f_{j,t}(\theta; \xi, \tau_{(<t)}) \mid \mathbf{G}_{j,t} - \mathbf{b}_{j,t}) \right]$$

$\nabla_{\theta} \log \pi$ is taken through every path in f : roles, gates, experts and teammates' bits. Roles are learned **pathwise**, through the reparameterized sample, not by a score-function term on p .

Why a role-dependent baseline is valid here

For any $b_{(j,t)}$ computed from information available before $a_{(j,t)}$ is sampled, which includes ξ and therefore p :

$$\mathbb{E}_{a \sim \pi_{\theta}(\cdot \mid f)} [\nabla_{\theta} \log \pi_{\theta}(a \mid f); b] = b, \quad \nabla_{\theta} \sum_a \pi_{\theta}(a \mid f) = b, \quad \nabla_{\theta} 1 = 0$$

So $V(h_s, \text{sg}(p), \kappa)$ is a valid baseline. The review's objection holds for a different estimator: if p were learned through $\nabla \log q(p \text{ given } H)$, a baseline depending on p would bias it, because $\mathbb{E}_p[\nabla \log q(p) b(p)] \neq 0$ in general. That estimator is not used.

Where the gradient is biased

Source	What happens	Status
Straight-through bits	$\partial b / \partial u$ is replaced by the derivative of the relaxed sample (section 5)	Biased; inherited from HetNet; must be stated

Source	What happens	Status
Truncated backpropagation	Terms linking steps more than 5 apart are dropped, in H , C , $\bar{u}$ and the prior's $p_{-}(t-1)$	Biased; inherited from HetNet's recipe
Per-agent returns	Agent j 's log-probability is weighted by its own return $G_{-}(j,t)$, not the team's, so effects of j 's action on teammates' rewards are ignored	A credit-assignment approximation inherited from HetNet, kept so the comparison is fair

The rate term's gradient is exact: the KL is closed-form and differentiated pathwise.

What this means for the implementation

HetNet's A2C keeps the rollout's computation graph and backpropagates once, which computes exactly this estimator with the realized ξ , provided that:

1. roles are drawn with a reparameterized sample and never detached on the actor path;
2. bits use the straight-through construction with the noise actually sampled;
3. nothing is recomputed between rollout and update.

Moving to PPO, with several epochs per batch, changes this. There you must store ξ and recompute each f with the stored noise, so the old and new probabilities refer to the same augmented state. Hard bits are then reproduced exactly at unchanged θ , because $b = \mathbf{1}[u + v > 0]$ is deterministic given v .

11. What the architecture does not guarantee

These limits hold by construction. Stating them up front keeps the claims honest and tells the experiments what they must show.

Not guaranteed	Why	What would show it
That roles are used	The rate term is minimized by $q = p$, which ignores history. A shared-weight network over H (ρ held constant) is always available, and among settings with equal return J_β prefers the one with the smaller rate. Roles carry information only if gate-selected processing buys return that this alternative does not, at equal capacity	Freezing or pinning the role changes behaviour and return (implementation guide, experiments)
That it contains HetNet	The banks are shared across agents; there is one attention budget instead of per-class ones; receivers get no sender-type side information, which HetNet's receivers get implicitly from the edge type	Not claimed. HetNet is a baseline
That adaptation happens	With θ frozen, $\rho_-(j,t) = f_\theta(H_-(j,t), \kappa_-(j,t), \varepsilon_-(j,t))$ still moves with inputs, so the effective weights change. That makes adaptation <i>representable</i> , not <i>achieved</i> . The training distribution must contain the events	Recovery after in-episode events, with no parameter updates
Performance on unseen team sizes	Shared node functions and set aggregation run on any team size, but attention distributions shift as neighbour counts change	Held-out compositions
That lost capability can be replaced	A role cannot create a sensor or an actuator. Recovery requires the surviving team to still hold the needed capabilities	Designing events where recovery is physically possible

Not guaranteed	Why	What would show it
Fault diagnosis	κ is given to the agent	Out of scope; a later variant could remove κ
That p means something discrete	g is a set of mixture weights over processing modes, not a probability over named roles. Separated clusters in a plot are not evidence	Interventions, not visualizations

Fallback objective, only if roles go unused

If the interventions show the role has no causal effect, add the other half of the information bottleneck. Roles must then predict the agent's own upcoming behaviour $Y_{(j,t)}$, such as its displacement or action histogram over the next k steps:

$$\begin{aligned} & \max_{\rho, \kappa} \mathbb{E} [I(\rho_t; \kappa_t, Y_t) - \beta \mathbb{E} [I(\rho_t; \kappa_t, H_t) \\ & \quad | \rho_{t-1}, \kappa_t] - \mathbb{E} [I(\rho_t; \kappa_t, Y_t) | \rho_t, \kappa_t]] \\ & \quad + \mathbb{E} [H(\rho_t | \kappa_t) + \mathbb{E} [\log q_{\psi}(\rho_t | Y_t, \kappa_t)]] \end{aligned}$$

This is the information-bottleneck Lagrangian (Tishby, Pereira and Bialek, 1999) with future behaviour as the target: a role becomes the compressed piece of history that predicts what the agent will do next. The inequality is the Barber–Agakov bound. In practice:

1. Train a discriminator q_{ψ} by maximum likelihood on (ρ_t, Y_t, κ_t) from rollouts.
2. Add $\eta \cdot \log q_{\psi}(\rho_{(j,t)} | Y_{(j,t)}, \kappa_{(j,t)})$ to agent j 's reward at step $t + k$, as DIAYN does. Behaviour must now depend on ρ , or the discriminator cannot recover it.
3. Sweep η small. This term rewards identifiable behaviour for its own sake and can compete with the task.

Caveat. $H(\rho_t | \kappa_t)$ is not constant. Replacing it by the prior's entropy is justified only while the marginal term of the rate identity, $\mathbb{E} \text{KL}(\bar{q} \parallel p)$, stays small, so monitor it.

12. Side by side, and the claim

	HetNet	ROMA	SoftRole-HetNet
Source of heterogeneity	Designer-given classes	Learned roles from the current observation	Learned roles from memory and known capabilities
Heterogeneity variable	Discrete, fixed per agent	3-d Gaussian, resampled each step	3-d Gaussian, resampled each step under a persistence prior
How it changes computation	Class-indexed matrices	Hypernetwork generates the Q-network's last layer	Gate mixes K weight sets in every communication round
Parameter sharing	Within a class	All agents	All agents, with K weight sets
Communication	n-bit binary, 2 rounds per step	None	n-bit binary, 2 rounds per step; roles never transmitted
Attention	Additive GAT (static); one softmax per sender class	None	GATv2 (dynamic); one softmax with a null slot
Memory of received messages	No (in the released code)	No messages	Yes, through $\bar{u}$
Action spaces	One output layer per class	Shared	Shared vocabulary with a feasibility mask
Learning	On-policy A2C, per-class critic	Off-policy QMIX	On-policy A2C, per-agent critic
Role regularizer	None	L_I (zero under its own model), L_D	KL rate to a persistence prior: a switching cost
Bandwidth per agent-step	$L \cdot n$ bits	0	$L \cdot n$ bits

The claim

Does a rate-limited adaptive role, conditioning both the binary channel and the agent's own processing, improve frozen-policy recovery after team or capability changes, beyond capacity-matched direct conditioning on history and capabilities?

Recent work shows why the claim must be this narrow. [Howell et al. \(CoRL 2023\)](#) generalize shared policies to new team compositions through capability awareness and capability communication. [CASH \(CoRL 2025\)](#) generates decoder weights with a hypernetwork over local observations, the robot's own capabilities and the team's capabilities, and evaluates online capability changes. [SCoUT \(2026\)](#) couples soft agent groups to recipient selection and to a group-aware critic. The review also cited a "GRDC" (AAAI 2026) that I could not find.

Judging from their abstracts, none of the three combines SoftRole's specific mechanism: a history-dependent, rate-limited latent that conditions both ends of a bandwidth-limited binary channel. That combination is what the experiments must isolate, and CASH-style direct conditioning is the baseline that matters most. CASH takes team capabilities as a direct input; in this setting that information has to travel through n bits, so a fair baseline gets the same channel, or the difference must be stated.

References

Built on

- Seraj et al. [Learning Efficient Diverse Communication for Cooperative Heterogeneous Teaming](#) (HetNet). AAMAS 2022.
- Wang, Dong, Lesser and Zhang. [ROMA: Multi-Agent Reinforcement Learning with Emergent Roles](#). ICML 2020.

Components and the results used

- Brody, Alon and Yahav. [How Attentive are Graph Attention Networks?](#) (GATv2; static vs dynamic attention). ICLR 2022.
- Jang, Gu and Poole. [Categorical Reparameterization with Gumbel-Softmax](#). ICLR 2017.
- Maddison, Mnih and Teh. [The Concrete Distribution](#). ICLR 2017.
- Bengio, Léonard and Courville. [Estimating or Propagating Gradients Through Stochastic Neurons](#) (straight-through). 2013.
- Kingma and Welling. [Auto-Encoding Variational Bayes](#) (reparameterization). ICLR 2014.

- Shazeer et al. [Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer](#). ICLR 2017.
- Perez et al. [FiLM: Visual Reasoning with a General Conditioning Layer](#). AAAI 2018.
- Hoffman and Johnson. ELBO surgery: yet another way to carve up the variational evidence lower bound. NeurIPS Workshop on Advances in Approximate Bayesian Inference, 2016.
- Alemi et al. [Deep Variational Information Bottleneck](#). ICLR 2017.
- Tishby, Pereira and Bialek. [The Information Bottleneck Method](#). 1999.
- Goyal et al. [InfoBot: Transfer and Exploration via the Information Bottleneck](#). ICLR 2019.
- Galashov et al. [Information asymmetry in KL-regularized RL](#). ICLR 2019.
- Ng and Jordan. [PEGASUS: A policy search method for large MDPs and POMDPs](#). UAI 2000.
- Schulman et al. [High-Dimensional Continuous Control Using Generalized Advantage Estimation](#). ICLR 2016.
- Barber and Agakov. The IM Algorithm: A Variational Approach to Information Maximization. NeurIPS 2003.
- Eysenbach et al. [Diversity is All You Need](#) (DIAYN). ICLR 2019.

Related work for the claim

- Howell et al. [Generalization of Heterogeneous Multi-Robot Policies via Awareness and Communication of Capabilities](#). CoRL 2023.
- Fu, Jain et al. [Capability-Aware Shared Hypernetworks for Flexible Heterogeneous Multi-Robot Coordination](#) (CASH). CoRL 2025.
- [SCoUT: Scalable Communication via Utility-Guided Temporal Grouping](#). arXiv, 2026.